PROJE 18 / 20 · ÇALIŞAN TEMEL SÜRÜM

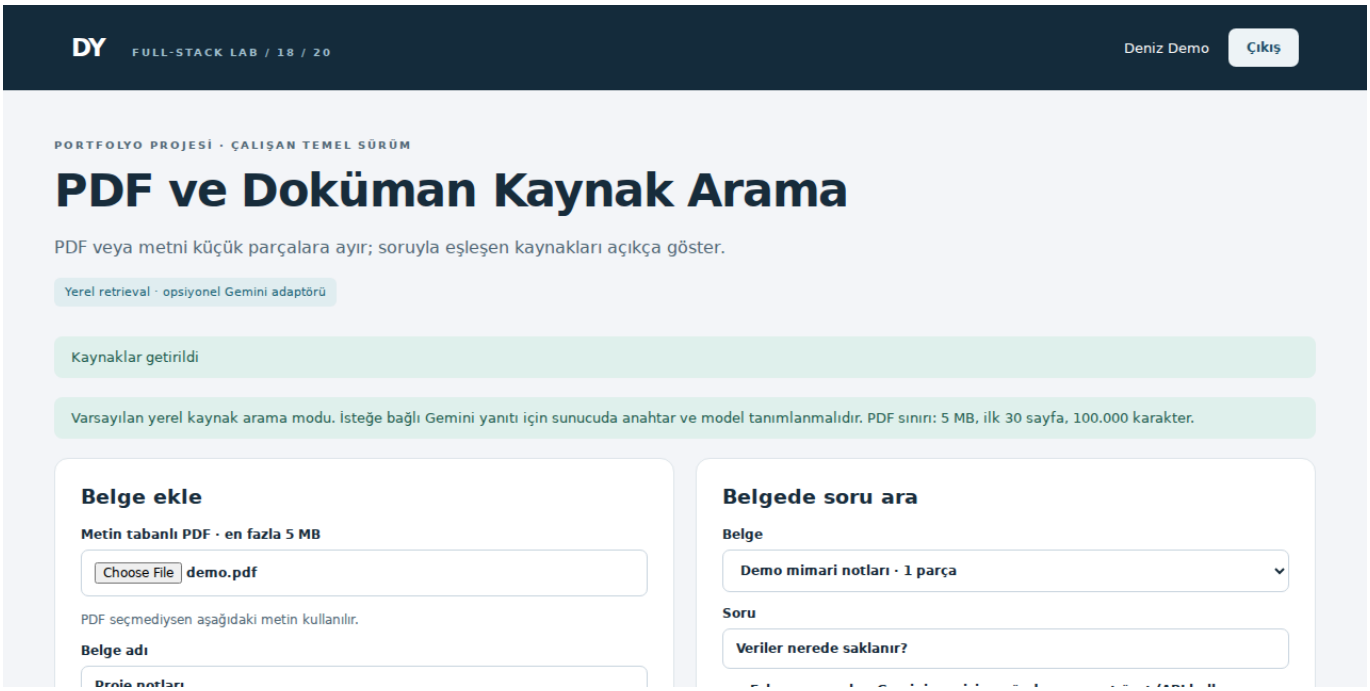
PDF ve Doküman Kaynak Arama

PDF veya metni küçük parçalara ayır; soruyla eşleşen kaynakları açıkça göster.

Yerel retrieval · opsiyonel Gemini adaptörü

Bu proje nedir?

Frontend kullanıcının gördüğü ekranları ve etkileşimleri, backend iş kurallarını ve veri erişimini yürütür. Bu uygulama ikisini aynı origin üzerinde birleştiren bağımsız bir full-stack örnektir. Üretim ürünü değil, üzerinde geliştirme yapabileceğin çalışan başlangıçtır.



Gerçek uygulama ekranı · yerel demo verileri

Teslim edilen kapsam

Metin/PDF yükleme · 800 karakterlik parçalar · 150 karakter örtüşme · Kelime eşleşmesiyle sıralama · Kaynak parça numarası

Bu sürümde olmayanlar

Varsayılan mod yapay zekâ yanıtı üretmez. GEMINI_API_KEY ve GEMINI_MODEL ile isteğe bağlı Gemini adaptörü vardır; anahtar olmadığı için canlı sağlayıcı çağrısı test edilmedi. Embedding, vektör arama ve OCR yok. PDF ilk 30 sayfa ve 100.000 karakterle sınırlıdır; limitler arayüzde açıklanır.

02 / Backend ve veri modeli

PDF metni parser ile çıkarılır; boş/taranmış PDF reddedilir. Metin 800 karakterlik pencerelerle 650 adımda bölünür. Sorunun üç ve üzeri karakterli kelimeleri tekilleştirilir. Her parça içerdiği farklı sorgu kelimesi sayısıyla puanlanır. Pozitif puanlı ilk üç parça döndürülür; kaynak yoksa cevap uydurulmaz.

Gerçek SQL şeması

```
CREATE TABLE documents(id TEXT PRIMARY KEY,owner TEXT REFERENCES users(id),name
TEXT,created_at TEXT);


 CREATE TABLE chunks(id TEXT PRIMARY KEY,document TEXT REFERENCES documents(id) ON DELETE
 CASCADE,position INTEGER,body TEXT);
```

Tam şema: schema.sql. users ve sessions ortak kimlik tablolarıdır. owner/user alanları kullanıcıyı, PRIMARY KEY kaydı, UNIQUE tekilliği ve REFERENCES ilişkiyi belirtir. Para kullanılan projelerde INTEGER kuruş, zaman alanlarında ISO metin veya epoch kullanılır.

Kod okuma sırası

Dosya	Ne arayacaksın?
client/Workspace.vue	Eylemi tetikleyen form ve API isteği
client/api.ts	JSON isteği, hata yakalama ve mesaj
server/project.js	Yetki kontrolü → kural → SQL → yanıt
server/core.js	Parametreli SQL, transaction, oturum, SSE

Neden bu mimari?

Aynı dilde frontend/backend öğrenmeyi kolaylaştırır. Node.js 24 ve Express HTTP katmanıdır; Vue arayüzü yönetir. SQLite ayrıca sunucu kurmadan kalıcılık sağlar. Senkron veritabanı işlemleri küçük yerel demo için seçildi; çok süreçli, yüksek trafikli üretim mimarisi iddiası yok.

03 / API sözleşmesi

Aşağıdaki yollar doğrudan teslim edilen server/project.js dosyasından çıkarılmıştır. JSON alan adları kodla aynıdır. :id ve benzeri path alanları oluşturma yanıtından alınmalıdır. Liste/görünüm uçlarında 200, yeni kayıt uçlarında 201, kuyruk kabulünde 202 kullanılır.

Yöntem / yol	Girdi alanları	Başarı
GET /api/documents	Gövde yok	200
POST /api/documents	JSON: name, base64, text	201
POST /api/documents/:id/ask	JSON: question, useLlm	200
DELETE /api/documents/:id	Gövde yok	200

Somut istek örneği

```
POST /api/documents
{"name":"Notlar","text":"SQLite verileri yerel dosyada ..."}
201 {"id":"<document-id>"}
```

Oturum ve hata biçimi

POST /api/auth/login: email ve password. Başarılı giriş HttpOnly çerez verir. GET /api/auth/me kullanıcıyı, POST /api/auth/logout çıkışı yönetir. Tarayıcı aynı origin kullanır. Genel hata biçimi: {"error":"açıklama"}. Yetkisiz alan işlemleri sunucuda engellenir; frontend kontrolü tek başına yeterli değildir.

Kod	Anlam
401 / 403	Giriş gerekli / rol veya origin reddedildi
404 / 409	Kayıt görünmüyor / durum veya benzersizlik çakışması
422 / 429	Geçersiz girdi / giriş veya uygulama kotası
500 / 502 / 503	Beklenmeyen hata / dış servis hatası / kapalı veya ayarsız servis

Tüm uçlar bütün kodları üretmez. İstek/yanıt gövdelerini tests/project.test.js ve scripts/seed.js içindeki çalışan örneklerle karşılaştır.

04 / Test ve kabul kriterleri

Bir projenin çalışması yalnız ekrana veri gelmesi değildir. Yetki, tekrar istek, yanlış girdi ve yarış koşulları da beklenen sonucu üretmelidir. Aşağıdaki maddeler teslim edilen temel sürümün kontrol listesidir.

- SQLite sorusu SQLite içeren kaynak döndürmeli.
- Eşleşmeyen soru boş kaynak listesi üretmeli.
- Başka hesabın belgesi sorgulanamamalı.
- Örnek metin tabanlı PDF metni çıkarılmalı; canlı Gemini çağrısı ayrıca denenmeli.

Otomatik test nasıl çalışır?

npm test, Node test runner ile gerçek HTTP sunucusunu rastgele yerel portta başlatır. İki kullanıcıyla oturum açar, uç noktalara istek gönderir ve durum/yanıtları assert ile denetler. Veritabanı :memory: olduğu için kendi data/app.sqlite dosyan etkilenmez. Projeye özgü testler tests/project.test.js içindedir.

```
npm ci
npm run typecheck
npm test
npm run build
```

Test kodundan örnek

```
const d=await post('/documents',{name:'Notes',text:'SQLite veritabanı yerel dosyada
saklanır. Vue arayüzü bu verilere Node API ile erişir.'},201);
const answer=await post('/documents/'+d.id+'/ask',{question:'SQLite
nerede?'});assert.equal(answer.mode,'local-
retrieval');assert.ok(answer.sources[0].text.includes('SQLite'));
const unknown=await post('/documents/'+d.id+'/ask',{question:'Galaksiler'});assert.equal
(unknown.sources.length,0);
await call('/api/documents/'+d.id+'/ask','POST',{question:'SQLite'},404,other);
```

Tarayıcıda kontrol et

Bir başarılı işlem, bir hatalı girdi ve boş durum göster. 390 px mobil görünümde yatay taşmayı kontrol et. Sayfayı yenilediğinde kalıcı kayıtlar durmalı. Network panelinde isteğin gövdesini ve dönen HTTP kodunu oku; başarısız işlemi başarılıymış gibi göstermemelisin.

Doğrulamanın sınırı

Canlı Gemini sağlayıcısı anahtar olmadığı için çağrılmadı. Yerel metin/PDF yolu ayrıca kontrol edildi.

05 / Çalışma ve GitHub planı

İlk günün somut hedefi

Metin yükle ve belgede geçen bir terimi sor. Geçmeyen bir kelime sorup boş kaynak sonucunu göster. Örnek PDF'i de yükleyerek parser yolunu dene.

Çalışma bloğu	Teslim edeceğin çıktı
1 · Kurulum / 30-60 dk	Çalışan ekran, örnek veri ve ilk başarılı API isteği
2 · Kod okuma / 45-90 dk	Formdan SQL sorgusuna kadar bir isteğin açıklaması
3 · Geliştirme / 60-120 dk	Bir iş kuralı veya arayüz davranışında kendi değişikliğin
4 · Doğrulama / 30-60 dk	İlgili test ve README değişikliği; yeni ekran görüntüsü

Süreler tahmindir; hata çözme ve yeni kavramlar daha uzun sürebilir. Bu plan hazır temel sürüm üzerinde çalışmak içindir. Projeyi sıfırdan ve üretim kalitesinde bir günde bitirme vaadi değildir.

İleri sürüm: dört gerçek geliştirme

- 1. Embedding ve vektör indeks ekle
- 2. Gemini adaptörünü kendi anahtarınla doğrula
- 3. Token bütçesi ve maliyet sınırı ekle
- 4. Prompt injection ve belge izin testleri ekle

Anlamlı commit sırası

```
git add .
git commit -m "chore: add document-search baseline"
# Kendi değişikliğini yaptıktan sonra:
git add client server tests README.md
git commit -m "feat: describe your actual improvement"
```

Yeni boş GitHub reposuna remote ekleyip push et. Burada gerçek hesap/repo adresi bulunmadığı için otomatik yükleme yapılmadı. node_modules, data ve .env dosyalarını hariç tut. Her gün bir gerçek ilerleme paylaş; commit sayısını artırmak için geçmiş üretme.

Mülakatta anlat

Kelime eşleşmesi ile anlamsal arama arasındaki fark nedir? Kaynak gösterilmesi modelin her iddiasını doğrular mı?

Özel kurulum / ilerleme notu

Yerel mod anahtarsız çalışır. Opsiyonel `.env`: GEMINI_API_KEY=... ve GEMINI_MODEL=hesabınızda erişilebilir model kimliği. `node --env-file=.env server/index.js --dev` ile başlat. Arayüzde Gemini seçeneğini işaretlersen yalnız eşleşen parçalar sağlayıcıya gönderilir. Canlı API çağrısı bu ortamda test edilmedi; ücret/kota sağlayıcı hesabına bağlıdır.